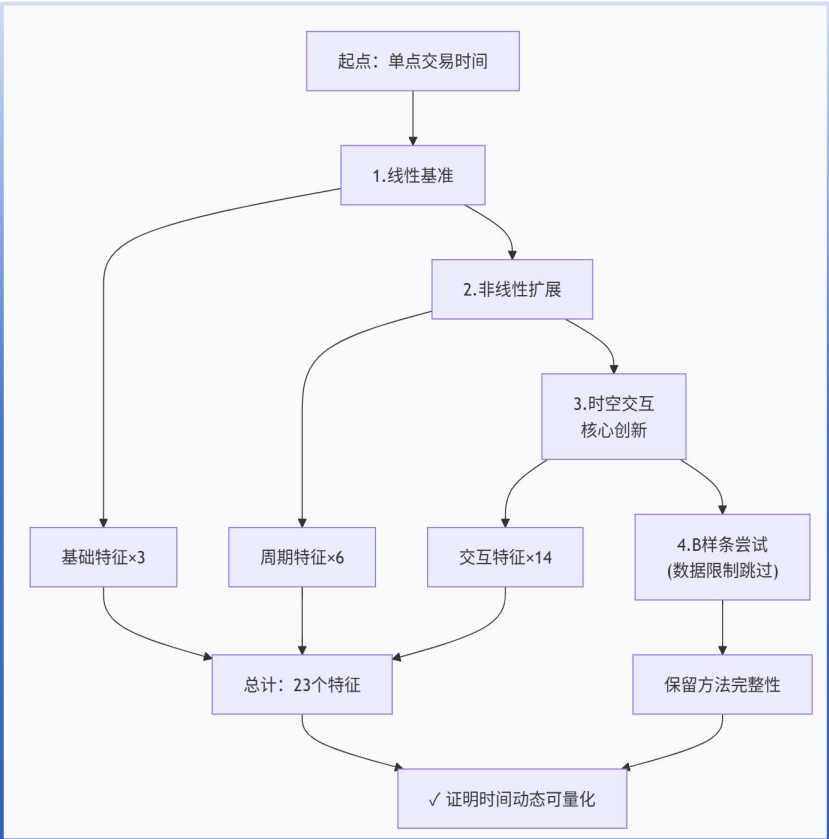


1. 房价预测中的市场时间场构建

基于单点交易时间的市场动态特征工程

量化市场敏感度



检查特征相关性...

时间序列特征与目标变量的相关性：

- 时间_连续月份：0.1362
- 时间_月度_sin：-0.0458
- 时间_月度_cos：0.0224
- 时间_是否旺季：0.0111
- 时间_是否季末：0.0268
- 时间_是否年初：-0.0070
- 时间_是否年底：0.0284
- 时间_是否上半年：-0.0502
- 时间_年份相对位置：0.1203
- 时间_是否近年：-0.0043

模型对比：

- 随机森林 R^2 ：0.9729
- XGBoost R^2 ：0.9672
- LightGBM R^2 ：0.9678

```
# ===== 1. 基础时间特征（基于训练集基准） =====
# 1.1 连续时间索引（使用训练集的基准）
df['时间_连续月份'] = (df['Transaction_Year'] - train_year_min) * 12 + (df['Transaction_Month'] - train_month_min)

(variable) df: DataFrame
df['时间_年份相对位置'] = df['Transaction_Year'] - train_year_mean

# 1.3 更精细的周期性特征
month = df['Transaction_Month']

# 使用sin/cos编码月份（比单独使用月份更好）
df['时间_月度_sin'] = np.sin(2 * np.pi * month / 12)
df['时间_月度_cos'] = np.cos(2 * np.pi * month / 12)

# 季节特征
df['时间_季节'] = (month - 1) // 3 + 1 # 1:冬, 2:春, 3:夏, 4:秋

# 业务周期特征
df['时间_是否旺季'] = month.isin([3, 4, 5, 9, 10]).astype(int) # 春秋旺季
df['时间_是否季末'] = month.isin([3, 6, 9, 12]).astype(int) # 季度末

# 特殊时间点
df['时间_是否年初'] = month.isin([1, 2]).astype(int)
df['时间_是否年底'] = month.isin([11, 12]).astype(int)
df['时间_是否上半年'] = (month <= 6).astype(int)
```

引入区域-时间交互项

```
if 'TE_City' in df.columns:
    df['交互_City_时间连续月份'] = df['TE_City'] * df['时间_连续月份']
    df['交互_City_年份相对位置'] = df['TE_City'] * df['时间_年份相对位置']

if 'TE_District' in df.columns:
    df['交互_District_时间连续月份'] = df['TE_District'] * df['时间_连续月份']
    df['交互_District_年份相对位置'] = df['TE_District'] * df['时间_年份相对位置']

    # 不同区域的季节性影响可能不同
    df['交互_District_月度sin'] = df['TE_District'] * df['时间_月度_sin']
    df['交互_District_月度cos'] = df['TE_District'] * df['时间_月度_cos']

if 'TE_Block_Stratified' in df.columns:
    df['交互_Block_时间连续月份'] = df['TE_Block_Stratified'] * df['时间_连续月份']
    df['交互_Block_年份相对位置'] = df['TE_Block_Stratified'] * df['时间_年份相对位置']

    # 区块级别的旺季效应可能不同
    df['交互_Block_是否旺季'] = df['TE_Block_Stratified'] * df['时间_是否旺季']
```

2. IDF-TF处理文本数据“核心卖点”

文本语义量化

```
# 定义TF-IDF提取器
class TfidfFeatureExtractor:
    """使用TF-IDF+SVD提取文本特征"""
    def __init__(self, n_components=8, max_features=200):
        """
        初始化TF-IDF特征提取器
        """
        print(f"使用TF-IDF+SVD降维到{n_components}维")
        # 创建处理管道
        self.pipeline = Pipeline([
            ('tfidf', TfidfVectorizer(
                max_features=max_features,
                stop_words=['的', '了', '在', '是', '有', '和', '与', '等', '等', '也', '都', '就'],
                token_pattern=r'[\u4e00-\u9fff]{2,}', # 匹配2个及以上中文字符
                min_df=3, # 至少出现3次
                max_df=0.9 # 最多出现在90%的文档中
            )),
            ('svd', TruncatedSVD(n_components=n_components, random_state=111))
        ])
        self.is_fitted = False
```

文本内容	处理方式	商业逻辑
“温馨舒适的家”	过滤	营销话术，无实际价值
“精装修拎包入住”	保留	装修成本可量化
“地铁口500米”	保留	交通便利可验证
“学区房名额在”	保留	教育资源溢价

3.基于R方与RMSE融合模型

手动加权融合

使用权重：LightGBM = 0.7, RandomForest = 0.3
手动融合性能：
RMSE：523837.8733
MAE ：230524.5910
R² ：0.955679
手动融合 RMSE 优于 LightGBM 单体。
文件已保存：price_submission_manual_blend.csv

数学思想

融合函数

$$\hat{y} = \sum_{i=1}^3 w_i \cdot f_i(X), \quad \text{其中} \sum_{i=1}^3 w_i = 1, \quad w_i \geq 0$$

权重优化目标

$$\min_{w_1, w_2, w_3} \frac{1}{N} \sum_{j=1}^N \left(y_j - \sum_{i=1}^3 w_i f_i(X_j) \right)^2$$

性能提升条件

$$\text{Var}(\hat{y}) \leq \max_i \text{Var}(f_i(X)), \quad \text{Bias}(\hat{y}) \approx \text{avg}_i \text{Bias}(f_i(X))$$

模型性能对比表

模型	In sample rmse	Out of sample rmse	Cross-validation rmse	Kaggle Score
期中线性模型	1029462.03	1031012.27	0.3350	65.8
随机森林	328,016.00	280,139.27	0.1324	75.2
XGBoost	515,751.65	496,442.54	0.1526	72.4
LightGBM	452,803.80	421,524.79	0.1359	75.4
ANN	846,049.89	827,681.43	0.2150	70.1

租金模型性能对比表（所有RMSE均为原始尺度）

模型	In sample rmse	Out of sample rmse	Cross-validation rmse	Cross-validation R ²	Kaggle Score
线性模型	302,367.40	292,782.73	297,575.07	0.8680	65.8
随机森林	230,786.16	234,309.71	234,309.71	0.9286	75.2
XGBoost	139,030.95	183,830.63	175,712.13	0.9200	72.4
LightGBM	164,310.46	159,662.12	183,300.08	0.9130	75.4
ANN	228,981.34	227,492.80	248,760.16	0.8396	70.1